

# Mémoire Technique

Alternant, développeur de logiciel interne

Sanofi Sisteron

45 Chem. de Meteline, 04200 Sisteron

MOULLET Grégory

THIERY Marc

FISA INFO 3A

## Sommaire

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>Sommaire.....</b>                                 | <b>1</b>  |
| <b>Introduction.....</b>                             | <b>2</b>  |
| <b>Présentation de l'entreprise.....</b>             | <b>3</b>  |
| <b>Présentation du site.....</b>                     | <b>3</b>  |
| <b>Présentation du service.....</b>                  | <b>5</b>  |
| <b>Comprendre React.....</b>                         | <b>6</b>  |
| Origine de React.....                                | 6         |
| Les bases de React.....                              | 7         |
| Notion de composant.....                             | 7         |
| Notion de Hooks.....                                 | 8         |
| Gestion du DOM virtuel.....                          | 10        |
| Cycle de vie d'un composant.....                     | 12        |
| <b>Pourquoi avoir choisi React.....</b>              | <b>13</b> |
| <b>Comment améliorer une application React.....</b>  | <b>14</b> |
| Utilisation de composants réutilisables et purs..... | 14        |
| Utilisation de Typescript.....                       | 16        |
| Architecture d'un projet.....                        | 17        |
| Les hooks plus poussés.....                          | 19        |
| Lazy loading.....                                    | 21        |
| Linter et Prettier.....                              | 22        |
| <b>Conclusion.....</b>                               | <b>23</b> |
| <b>Bibliographie.....</b>                            | <b>25</b> |

## Introduction

En alternance depuis maintenant presque deux ans chez **Sanofi**, sur le site de **Sisteron**, j'ai progressivement découvert le fonctionnement de mon service, les différentes méthodes de travail ainsi que les technologies utilisées. Depuis quelques années, les standards technologiques du service évoluent. En effet, auparavant, la quasi-totalité des applications développées étaient des **applications de bureau**, créées principalement en **C#** ou avec **Datagraph**, un framework interne au service.

Mais plus récemment, avec l'essor des technologies web et les nombreux avantages qu'elles offrent par rapport aux applications de bureau notamment en termes de **déploiement** et **d'accessibilité**, de plus en plus d'applications du service sont désormais développées en technologies **web**. Reste alors à faire un choix structurant quelles technologies adopter, aussi bien côté frontend que backend ? De nombreuses solutions ont été expérimentées au fil des années : PHP, JavaScript, EJS, TypeScript, Vue, React, Next.js, Node.js, etc.

Concernant la partie **frontend**, l'équipe a finalement orienté sa préférence vers **React**, une technologie aujourd'hui largement adoptée dans l'industrie, que nous détaillerons plus loin. L'objectif est désormais de faire de React un **standard technologique du service**, afin d'éviter la nécessité de devoir maîtriser une multitude de technologies différentes, ce qui complexifie le développement et la maintenance. Maintenant que ce choix technologique est posé, une question majeure se pose : Quels moyens techniques et architecturaux permettent de rendre une application React plus performante, plus maintenable, et moins coûteuse à exécuter ?

Dans ce mémoire technique, nous commencerons par une présentation de l'entreprise. Nous enchaînerons sur la présentation de React, de son fonctionnement, et des raisons qui ont motivé son adoption dans notre service, au détriment d'autres solutions. Ensuite nous parlerons de pourquoi avoir choisi React comme technologie au sein de notre service. Dans une autre partie, nous dresserons une liste détaillée de bonnes pratiques et notions poussées visant à optimiser le développement d'une application React, tant sur le plan technique, architectural, que sur la maintenabilité et des performances

## Présentation de l'entreprise

Dans cette première partie du développement, nous allons présenter l'entreprise Sanofi dans son ensemble, puis parler plus spécifiquement du site situé à Sisteron, avant d'examiner en détail le service où j'effectue mon alternance.

Sanofi est l'une **des plus grandes entreprises pharmaceutiques mondiales**, engagée dans la **recherche**, le **développement**, la fabrication et la commercialisation de **produits pharmaceutiques**. Fondée en **1973**, la société a son siège en France et opère à l'échelle mondiale, avec des activités dans plus de **170 pays**. Sanofi se concentre sur plusieurs domaines thérapeutiques clés tels que les maladies cardiovasculaires, les maladies métaboliques, les maladies rares, l'oncologie, les vaccins et les médicaments en vente libre.

Sanofi se positionne comme une entreprise intégrée, couvrant toutes les étapes du développement et de la commercialisation des médicaments, de la recherche préclinique à la production en passant par les essais cliniques. La société propose une vaste gamme de produits pharmaceutiques, comprenant des médicaments innovants et des traitements essentiels pour répondre aux besoins de santé de millions de personnes à travers le monde.

## Présentation du site

Je réalise mon alternance au sein du site de production situé à **Sisteron**, dans la région Provence-Alpes-Côte d'Azur. Ce site appartient à la filiale Sanofi Chimie.

Le site de Sisteron a été créé en **1917** pour répondre aux besoins de défense nationale par la société Allais et Forge. Il a ensuite été utilisé pour la production de poudre pendant la Seconde Guerre mondiale. En 1973, le site est devenu membre de l'entité Sanofi, mais ce n'est qu'en 1993 que la production de médicaments a commencé. En 1986, le site de Sisteron est devenu membre de la filiale Sanofi Chimie, et depuis lors, il se développe au même rythme que l'entité et compte actuellement plus de **600 employés**.



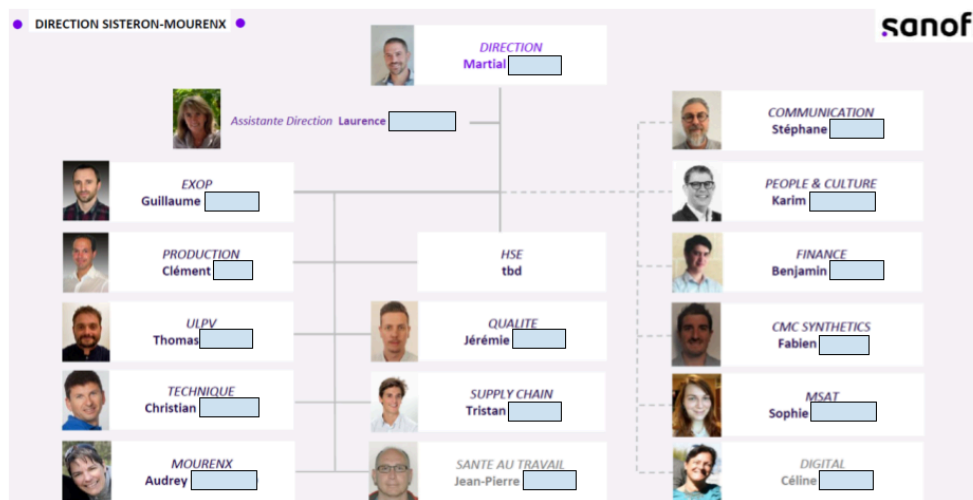
Photo panoramique de l'usine Sanofi Chimie à Sisteron; Source: Sanofi.fr

Au sein du site, deux principales activités sont menées :

La **Production** : Cette activité vise à fabriquer les **principes actifs** nécessaires aux usines de conditionnement de l'entité. Elle implique la mise en œuvre de procédés de fabrication rigoureux et de contrôles de qualité pour assurer la production efficace et conforme aux normes.

La **Recherche et Développement des procédés (R&D)** : Cette activité a pour objectif de développer et d'améliorer les procédés de fabrication des molécules issues de la recherche, en les adaptant aux grandes lignes de production à l'échelle industrielle. La R&D cherche à optimiser les procédés existants, à explorer de nouvelles techniques et technologies, et à garantir la sécurité, la qualité et l'efficacité des produits.

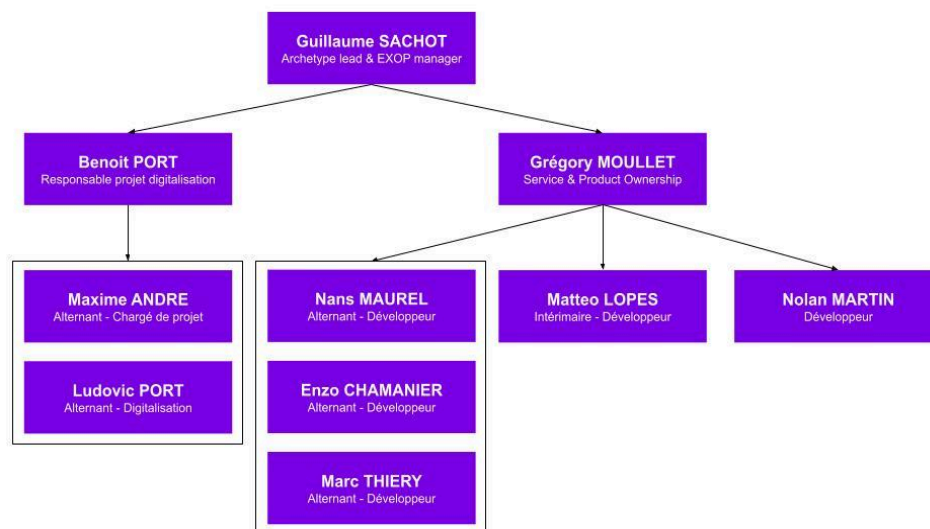
Pour assurer le bon fonctionnement de l'usine, différents services sont présents. Voici les responsables de chaque service, représentés sous forme d'organigramme :



Organigramme direction Sisteron-Mourenx; Source: document interne

## Présentation du service

Le service dans lequel je réalise mon alternance est l'**Excellence Opérationnelle** (EXOP), dirigé par Guillaume Sachot. Ce service est important au sein de l'entreprise, car il est en relation avec la majorité des autres services. Il se divise en deux parties distinctes. Nous allons donc nous pencher sur l'équipe au sein de laquelle on m'a affecté



J'ai travaillé au sein de la partie "**Digitalisation**" d'EXOP. La mission de ce service est de **développer des applications et des outils en réponse aux besoins exprimés par les autres services de Sanofi Sisteron**. C'est dans ce service que sont centralisées toutes les propositions de digitalisation provenant de l'ensemble de l'usine. L'équipe est principalement composée d'alternants, chacun étant responsable d'un ou plusieurs projets et travaillant à plusieurs ou seul sur le projet. Deux ingénieurs, Gregory Moullet et Benoît Port, encadrent cette équipe d'alternants. Gregory Moullet, ingénieur en génie industriel spécialisé en mécanique, a été mon maître d'apprentissage et celui qui m'a accueilli au sein de l'équipe.

## Comprendre React

### Origine de React

**React** est une bibliothèque **JavaScript open-source** orientée **frontend**, c'est-à-dire qu'elle agit sur la partie **interface utilisateur** d'une application. Elle permet de créer des applications web modernes, notamment grâce à son système de **Single Page Application** (SPA), qui repose sur une seule et unique page HTML. À chaque changement de contenu, on ne recharge pas la page : on modifie dynamiquement l'affichage à l'intérieur de la page existante. Cela permet d'offrir une expérience utilisateur **fluide et réactive**, sans interruption liée à un rechargement complet.

React a été développé par **Meta** en **2011**, par l'ingénieur **Jordan Walke**, dans le but de **simplifier la maintenabilité** des applications web, devenues de plus en plus **complexes et volumineuses** avec le temps.

Plusieurs **problèmes fondamentaux** ont motivé la création de React :

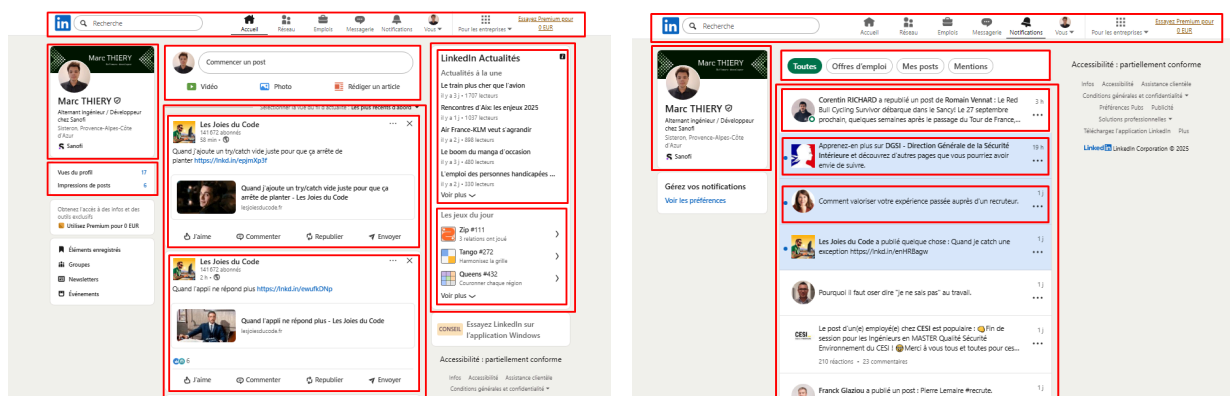
- La **multiplication des interactions** utilisateur à gérer dans les interfaces modernes
- La **lourdeur des manipulations du DOM** (un concept que nous détaillerons dans la suite)
- La **faible réutilisabilité** du code lié à l'interface utilisateur

Nous allons donc, dans la partie suivante, explorer les fondements de React et voir comment cette bibliothèque répond concrètement aux problèmes mentionnés ci-dessus.

## Les bases de React

### Notion de composant

Commençons tout d'abord par une des notions centrales faisant de React une des nouvelles technologies Web : la notion de **composant**. Un composant React peut s'apparenter à un bloc qui va contenir du code et que l'on va pouvoir **réutiliser** tout au long de l'application, dans les différentes pages. Un composant peut désigner un élément interactif, comme un simple bouton ou un champ. Il peut désigner quelque chose de plus complexe, comme un formulaire ou une liste dynamique. Il peut aussi désigner un élément visuel, comme une barre de navigation. Toujours dans le but de segmenter les pages en composants réutilisables, voyons réellement à quoi cela ressemble dans une application, où nous allons prendre comme exemple *LinkedIn*.



Captures d'écran de LinkedIn

Sur les deux captures d'écran ci-dessus, on retrouve, entourés en rouge, les composants. Comme on peut le voir au niveau du profil, sur les pages « Accueil » et « Notifications », on retrouve le même bloc / le même composant. Cela montre la réutilisabilité qu'offrent les composants, car ce morceau d'affichage n'a été écrit qu'une seule fois, mais utilisé deux fois. On retrouve ce même exemple avec la barre de navigation, mais ici, une chose change : la page sélectionnée dans la navbar. En React, on peut faire passer des propriétés appelées « **props** » afin de transmettre, depuis l'endroit où l'on appelle le composant, une ou plusieurs variables à ce même composant. Ce faisant, on peut faire varier le comportement selon ce que l'on passe en **props** à notre composant.

```
Composant Message.jsx
function Message(props) {
  return <p>Bonjour, {props.nom} !</p>;
}

Page App.jsx
function App() {
  return (
    <div>
      <Message nom="Alice" />
      <Message nom="Bob" />
    </div>
  );
}
```

Dans le morceau de code ci-dessus, on retrouve le composant **Message**, qui prend en props un nom et affiche le texte « Bonjour » suivi du nom passé dans les props. Plus bas, on retrouve la page **App** qui appelle le composant **Message** deux fois, en lui passant deux noms différents via les props. À l’affichage, on verra : « Bonjour, Alice Bonjour, Bob ».

Pour finir sur le fonctionnement des composants, si l’on retourne sur les captures d’écran au-dessus, on voit deux fils d’actualités. Ces fils sont des composants, mais contiennent aussi d’autres composants (les posts d’un côté et les notifications de l’autre). En React, il est donc également **possible qu’un composant utilise d’autres composants**, augmentant ainsi encore davantage la réutilisabilité et la division du code, ce qui rend ce dernier plus **structuré** et **organisé**. Avec cette notion de composant, on répond à la **troisième contrainte** énoncée plus tôt dans les origines de React.

## Notion de Hooks

Voyons maintenant une autre notion majeure de React : les **hooks**. Les **hooks** sont, globalement, des outils qui permettent d’accéder à **l’état** et aux **fonctionnalités** de React sans passer par des classes. Voyons ce que cela signifie en présentant les deux hooks les plus courants.

- **useState**

Ce hook permet de conserver une valeur passée à l'initialisation et de la maintenir entre les mises à jour de la page. Si l'on met à jour la valeur du **useState**, cela provoque une mise à jour de l'affichage, et l'on verra directement la nouvelle valeur à l'écran.

```
import React, { useState } from 'react';

function Compteur() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Vous avez cliqué {count} fois</p>
      <button onClick={() => setCount(count + 1)}>
        Cliquez ici
      </button>
    </div>
  );
}
```

On retrouve ici une implémentation simple de **useState**. On déclare le **useState** en haut de notre composant avec deux variables : **count**, la valeur du compteur initialisée à 0, et **setCount**, qui permet de mettre à jour la valeur de **count** afin de provoquer une mise à jour à l'écran. Plus bas, dans le **return**, on retrouve une balise affichant la valeur du compteur (0 au début) et un bouton qui va incrémenter cette valeur. Là où, en JavaScript natif, il aurait fallu utiliser **getElementById** pour cibler notre balise **<p>** et y modifier le contenu, ici, cela se fait automatiquement.

- **useEffect**

Ce hook permet de gérer les **effets de bord**, c'est-à-dire tout ce qui est **externe** ou **asynchrone** par rapport au rendu du composant. On peut citer, comme exemples simples, un **appel à une API** ou encore la gestion d'un timer.

```
import { useState, useEffect } from 'react';

const [users, setUsers] = useState([]);
useEffect(() => {
  fetch('https://getListUsers')
    .then(res => res.json())
    .then(data => {
```

```
setUsers(data);  
});  
, []);
```

Dans cette implémentation de code, on cherche à récupérer la liste de nos utilisateurs via une API distante. On commence par définir un **useState** pour stocker l'état de la liste des utilisateurs. Ensuite, le **useEffect** se lance automatiquement au chargement du composant, car il possède un **tableau vide en argument** (dernière ligne).

À l'intérieur du **useEffect**, on retrouve un appel API qui, une fois le résultat reçu, met à jour la liste des utilisateurs à travers le **useState**. Là où le **useEffect** devient intéressant, c'est qu'il est possible de modifier sa façon d'être déclenché. En effet, comme mentionné précédemment, si l'on passe un tableau vide en paramètre, le **useEffect** ne s'exécute qu'une seule fois, au chargement du composant.

Mais si l'on ne passe **aucun tableau**, le **useEffect** se relance à chaque mise à jour du composant. Enfin, on peut également passer des **dépendances** dans ce tableau (variables/fonctions): cela aura pour effet de relancer le **useEffect** à chaque fois qu'une dépendance est modifiée. C'est particulièrement utile, par exemple, si l'on modifie dynamiquement la liste des utilisateurs.

Nous avons pu voir ici deux exemples de hooks, mais il en **existe une multitude** utilisable et intégré à React où nous en présenterons quelques uns plus poussé quand nous parlerons optimiser. De plus, il est possible de créer ses propres hooks, une bonne pratique que nous détaillerons dans la suite.

Nous avons parlé dans les paragraphes précédents de mise à jour sur la page mais qu'est ce que cela veut vraiment dire ? Nous allons voir dans la sous partie suivante le fonctionnement du **DOM**, un composant essentiel du web ainsi que le **DOM virtuel** géré par React.

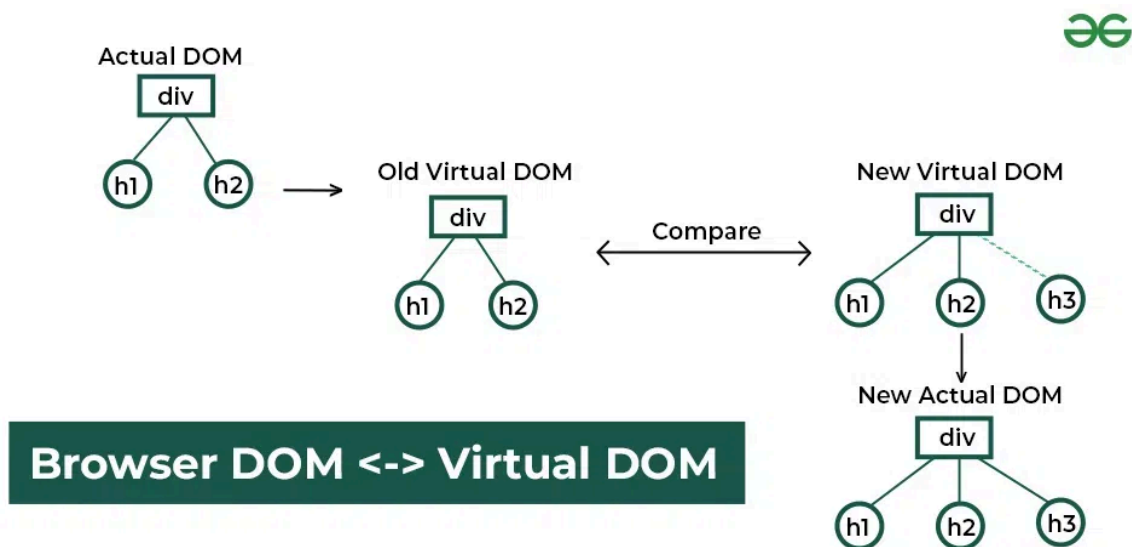
## Gestion du DOM virtuel

Pour commencer, parlons du **DOM**. Le DOM (Document Object Model) est une représentation en mémoire de la structure HTML d'une page web. Il prend la forme d'un arbre d'éléments appelés **nœuds**. Ces nœuds sont manipulables via JavaScript et leur modification impacte directement le DOM, provoquant une mise à jour de l'affichage. Toutefois, lorsque le DOM est utilisé de manière **intensive**, **cela peut entraîner des lenteurs**.

C'est là qu'intervient le **DOM virtuel** de React. Le DOM virtuel, ou *VDOM*, est une représentation légère du DOM. « Il permet à React de gérer plus efficacement les mises à jour de l'interface utilisateur en conservant une version virtuelle de celle-ci en mémoire. » [DOM virtuel ReactJS - GeeksforGeeks](#)

Le **VDOM** commence par **copier le DOM réel**. Lorsqu'un état est modifié (par exemple via un `useState`), React crée un nouveau DOM virtuel contenant cette modification. Il **compare** ensuite l'ancien et le nouveau VDOM pour identifier l'ensemble minimal de changements à effectuer. Enfin, React **applique uniquement ces modifications au DOM réel**, ce qui **optimise** considérablement les performances de rendu.

De plus, la gestion du rendu et des mises à jour se fait automatiquement grâce à ce mécanisme, **réduisant ainsi la quantité d'interactions manuelles à gérer**. Avec cette notion de VDOM, on répond à la **première** et à la **deuxième contrainte** évoquées plus tôt dans les origines de React.



Fonctionnement de la mise à jour du DOM avec le VDOM; source : [DOM virtuel ReactJS - GeeksforGeeks](#)

## Cycle de vie d'un composant

Dans cette dernière partie des bases de React, nous allons parler du **cycle de vie d'un composant**. Il est important de noter que le cycle de vie présenté ici ne s'applique qu'aux projets React à **partir de la version 16.8**, sortie en **2019**. En effet, à partir de cette version, React a profondément revu la notion de composants, en adoptant les **composants fonctionnels** à la place des **composants de classe**, avec l'introduction des **hooks**. L'objectif était de rendre le code plus **réutilisable** et d'offrir de nouvelles fonctionnalités, que nous verrons plus en détail par la suite.

Nous allons donc présenter le cycle de vie d'un composant à travers l'exemple du hook **useEffect**. Ce cycle se divise en **trois phases** : le **montage**, la **mise à jour** et le **démontage**.

- La première phase, **le montage**, correspond au moment où React installe le composant et effectue le rendu initial via le VDOM. Dans un **useEffect**, c'est ce comportement qui se produit lorsque l'on passe **un tableau vide** en argument : le hook ne s'exécute qu'une seule fois, au chargement.
- La deuxième phase, **la mise à jour**, se déclenche lorsqu'un changement d'**état** ou de **props** survient. Par exemple, si l'on place une variable dans le tableau de dépendances du **useEffect**, celui-ci se relancera à chaque fois que cette variable change, déclenchant ainsi une mise à jour du VDOM.
- La dernière phase, **le démontage**, intervient lorsqu'un composant est retiré du VDOM, comme lors d'un changement de page ou d'une condition qui le rend inactif. Dans le **useEffect**, on peut gérer cette phase en retournant une fonction de nettoyage qui sera exécutée juste avant le retrait du composant.

## Pourquoi avoir choisi React

Parlons à présent des raisons concrètes qui nous ont poussés à accentuer le développement autour de **React** dans notre service, au détriment d'autres frameworks web comme **Vue** ou **Angular**, ou encore de technologies plus anciennes.

Premièrement, cela tient à la **connaissance des développeurs** au sein du service. La plupart des membres de l'équipe viennent d'horizons différents et n'ont pas été formés aux technologies plus anciennes sur lesquelles reposent certains outils récents. De ce fait, le JavaScript natif a été rapidement écarté.

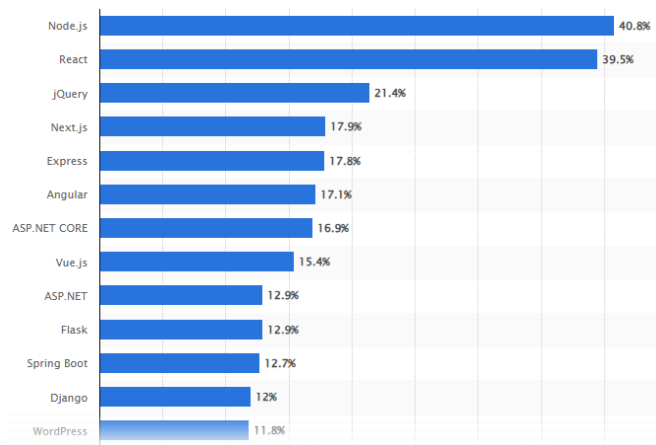
Parmi les trois grands frameworks **React**, **Vue** et **Angular**, peu de développeurs étaient familiers avec Angular, et parmi ceux qui l'étaient, la majorité critiquait son **aspect trop rigide**. Il restait alors deux candidats : React et Vue.

Il est à noter qu'un **standard technologique** au sein de Sanofi recommande l'utilisation de Vue, notamment pour **favoriser le partage de composants** entre les différentes équipes. Toutefois, les échanges entre équipes sont quasi inexistant, ce qui limite la pertinence de cet argument dans notre contexte.

Sur le plan technique, **les deux frameworks ont été jugés équivalents**. C'est donc l'environnement **autour** de la technologie qui a orienté notre choix.

En effet, React est soutenu par **Meta**, ce qui garantit une **solide maintenabilité à long terme**, des mises à jour régulières et une forte communauté. À l'inverse, Vue repose sur une équipe plus restreinte de développeurs et de bénévoles, ce qui, même si la solution est stable, peut soulever des questions à long terme.

De plus, en ce qui concerne **la valeur ajoutée pour les développeurs**, maîtriser React semble être un **atout professionnel plus fort**. C'est une compétence largement demandée sur le marché, et elle renforce notre **employabilité**.



Enfin, en observant le graphique ci-contre, on constate que **React est de loin la technologie frontend la plus utilisée**, devant tous ses concurrents.

Nous avons donc estimé que, **en termes de stratégie et de bagage professionnelle**, React est la meilleure technologie à adopter et à standardiser au sein du service.

% d'utilisations des technologies web par les développeurs; source : [Most used web frameworks among developers 2024 | Statista](#)

## Comment améliorer une application React

Dans cette troisième partie, après avoir décrit les points essentiels de React pour bien comprendre son fonctionnement, nous allons aborder les **bonnes pratiques** pour développer une application React, ainsi que des **notions moins connues** permettant **d'améliorer** une application dans certains cas.

### Utilisation de composants réutilisables et purs

Dans React, la notion de composant est centrale, et c'est en développant correctement ces derniers que l'on gagne énormément en **maintenabilité** et en **scalabilité**. En effet, plus on divise l'application en composants, plus on peut réutiliser ces mêmes composants tout au long de l'application, créant ainsi un standard, tant sur l'interface que sur le code. Cela **réduit** considérablement la

**quantité de code à maintenir**, et lorsqu'un bogue survient, sa correction se répercute automatiquement partout où le composant est utilisé. De même, pour la partie visuelle, si l'on ajoute une donnée dans un composant ou que l'on modifie un style, cela s'appliquera systématiquement **partout**.

Pour prendre un exemple simple illustrant la puissance d'un **composant réutilisable**, on peut créer un composant **Formulaire**. Ce composant prend en **props** une action à exécuter lors de la soumission ainsi qu'une liste de champs. En fonction du type de champ, le composant **Formulaire** adapte sa logique et génère **différents types de champs, eux-mêmes composants**. Une fois cela implémenté, tous les développeurs travaillant sur le projet n'auront qu'à appeler ce composant en passant l'action et la liste de champs, et tout fonctionnera automatiquement, tout en gardant un **standard homogène**.

Le grand avantage des composants réutilisables est qu'à la différence des technologies plus classiques, où le temps de développement augmente avec la taille de l'application, ici le **temps de développement diminue progressivement**, puisque les composants sont déjà créés.

Passons maintenant à la notion de **composant pur**. Un composant pur est un composant qui **dépend uniquement de ses props**. Il est important de comprendre un comportement de React avant d'aller plus loin : **lorsqu'un composant parent est modifié, un re-render (mise à jour de l'interface virtuelle) est déclenché, et tous les enfants de ce parent se re-render également**.

Prenons un exemple simple de code pour illustrer cela :

```
import { useState } from "react";

function Parent() {
  const [count, setCount] = useState(0);
  console.log("parent re-render")
  return (
    <div>
      <h2>Parent</h2>
      <button onClick={() => setCount(count + 1)}>Ajouter 1</button>
      <Child nom="Alice"/>
    </div>
  );
}

function Child(props) {
  console.log("child re-render")
  return <p>Je suis l'enfant {props.nom}</p>;
}
```

Dans ce morceau de code, si on l'exécute et que l'on clique sur le bouton pour incrémenter, on verra en console le message « parent re-render » car le composant Parent a subi une mise à jour d'état. On verra aussi apparaître « child re-render ». Ici, le composant **Child n'est pas un composant pur** puisqu'il ne dépend pas uniquement de ses props. Il est donc **illogique que l'enfant se re-render alors que ses props n'ont pas changé**, ce qui entraîne une **perte de performance**. Dans cet exemple, il ne s'agit que d'un simple texte, mais si le composant enfant affichait une liste déclenchant un `useEffect` pour récupérer des données distantes, cette perte de performance serait bien plus importante.

Heureusement, il existe une solution : une fonctionnalité disponible uniquement pour les composants fonctionnels, comme ceux que nous utilisons, appelée **React.memo**.

**React.memo** permet d'**empêcher un re-render inutile** si les props du composant n'ont pas changé. En effet, **React.memo** mémorise les props du rendu précédent et les compare à ceux du rendu actuel. S'il n'y a pas de différence, il ignore la mise à jour. Sinon, il re-render. Cette technique d'optimisation s'appelle la **mémoïsation**.

## Utilisation de Typescript

React utilise par défaut JavaScript comme langage de programmation. C'est un langage simple d'utilisation, mais il présente un **défaut important**, particulièrement gênant lorsqu'une équipe travaille sur un projet : le typage. En effet, JavaScript n'est pas typé, c'est-à-dire que l'on ne précise pas si une variable est un entier, une chaîne de caractères, etc. De plus, la **gestion des classes et des objets est assez limitée**, ce qui peut conduire à un code difficile à maintenir au fil du temps, car il est compliqué de savoir rapidement ce que contiennent les objets et variables.

C'est justement pour cette raison que **TypeScript** a été créé. Avec TypeScript, on **renforce la qualité du code** en imposant un typage strict. Ainsi, le code devient beaucoup plus **maintenable**, car on sait précisément quelles données transitent, par exemple après un appel API, et quel est le format attendu.

TypeScript permet de définir des **structures de données précises** pour représenter ces formats. Une mauvaise pratique courante souvent suggérée par des outils comme ChatGPT est d'utiliser des **interfaces** pour typer ces formats. Pourtant, une interface peut aussi contenir des fonctions, ce qui n'est pas nécessaire dans ce contexte.

En TypeScript, il est préférable d'utiliser ce que l'on appelle des **type**, qui ne servent qu'à décrire la forme des données, sans fonctions ni méthodes.

```
type User = {  
  id: number;  
  name: string;  
};  
  
const user: User = {  
  id: 1,  
  name: "Alice"  
};
```

## Architecture d'un projet

Il est maintenant intéressant de se pencher sur la **manière d'architecturer** un projet React afin de le rendre facilement **maintenable** et **scalable**. Voici un exemple d'architecture bien organisée, divisant clairement les responsabilités.

À la racine du projet, on retrouve les fichiers de configuration ainsi que le fichier **.env** pour stocker les données sensibles.

Ensuite, deux dossiers principaux partent de la racine :

- **public**, qui contient les ressources accessibles à tous, comme le favicon
- **src**, qui est le dossier principal où se trouve la majeure partie du code

À la racine de **src**, on trouve le fichier **main.tsx**, point d'entrée de l'application, qui lance **App.tsx**. Ce dernier gère le routage de l'application, c'est-à-dire la navigation entre les différentes pages selon l'URL.

Les pages elles-mêmes sont situées dans le dossier **pages**, avec par exemple **Home.tsx**. Cette page utilise des hooks, comme **useGetListUser.ts**, pour récupérer la liste des utilisateurs.

Ce hook, basé sur **useEffect**, fait appel à un service, ici **userService.ts** qui contacte une API distante afin de rafraîchir son état.

Le résultat est une liste de type **User**, un **type** défini dans le dossier **types**.

Une fois les données récupérées, la page **Home** affiche la liste des utilisateurs à l'aide de composants : **ListUser.tsx**, qui lui-même appelle une liste de composants **UserCard.tsx**.

Cette organisation permet de bien séparer les responsabilités, chaque fichier ayant un rôle précis, ce qui est une règle fondamentale pour assurer la **qualité** et la **maintenabilité** du code.

```
projetReact/  
├── public/  
├── src/  
│   ├── assets/  
│   │   ├── images/  
│   │   ├── icons/  
│   │   └── fonts/  
│   ├── components/  
│   │   ├── Inputs/  
│   │   │   ├── TextInput.tsx  
│   │   │   └── SelectInput.tsx  
│   │   └── UsersShow/  
│   │       ├── ListUser.tsx  
│   │       └── UserCard.tsx  
│   ├── pages/  
│   │   ├── Home.tsx  
│   │   └── Login.tsx  
│   ├── hooks/  
│   │   ├── useGetListUser.ts  
│   │   └── useAuth.ts  
│   ├── services/  
│   │   ├── userService.ts  
│   │   └── authService.ts  
│   ├── styles/  
│   │   └── globals.css  
│   ├── types/  
│   │   └── User.ts  
│   ├── App.tsx  
│   └── main.tsx  
├── .env  
├── tsconfig.json  
└── package.json
```

Dans cette architecture, nous avons créé des **hooks personnalisés**, voyons dans la prochaine partie l'intérêt de ces hooks, ainsi que d'autres hooks intégrés à React, importants à connaître, qui peuvent **significativement améliorer** les **performances** d'un projet.

## Les hooks plus poussés

Pour commencer cette partie, parlons des **hooks personnalisés**. Il y a trois grands avantages qui font de leur utilisation une bonne pratique.

**Premièrement**, cela permet de **factoriser le code**. En effet, si l'on utilise un **useEffect** identique à plusieurs endroits du projet. Par exemple, pour appeler la même route d'API, il est préférable de déplacer cette logique dans un hook personnalisé, puis de simplement l'appeler dans les différentes parties concernées.

**Deuxièmement**, cela rend le code des pages et des composants **beaucoup plus lisible**. En divisant mieux la logique, les fichiers deviennent moins volumineux et plus faciles à lire et à maintenir.

**Enfin**, cela permet **d'encapsuler la logique métier**. En externalisant certaines parties fonctionnelles, les pages et composants peuvent se concentrer uniquement sur l'affichage, tandis que les hooks personnalisés gèrent des aspects comme l'authentification, la récupération ou l'envoi de données.

Passons maintenant à la présentation des **hooks liés à l'optimisation**.

Dans cette section, nous allons aborder deux hooks essentiels : **useMemo** et **useCallback**, qui permettent tous deux de mieux contrôler les effets de re-render et d'améliorer les performances de l'application.

- **useMemo**

Ce hook ressemble beaucoup à **React.memo**, présenté plus tôt. Il permet d'empêcher de relancer un calcul à chaque rendu lorsque ce calcul est trop coûteux. Comme pour **React.memo**, on lui fournit une/des dépendances qui détermine quand le calcul doit être relancé, par exemple si les données d'entrée ont changé.

```
const resultat = useMemo(() => {
  let total: int = 0;
  for (let i: int = 0; i < 100000; i++) {
    total = total + i + nombre;
  }
  return total;
}, [nombre]);
```

Dans cet exemple ci-dessus, c'est seulement si le nombre change dans que l'on va relancer le calcul.

- **useCallback**

Quand un composant se re-render à cause d'un changement d'état, tout à l'intérieur est également re-render, y compris les fonctions.

Dans certains cas, certaines fonctions sont très gourmandes, et l'idée de devoir recréer ces fonctions alors qu'elles ne sont pas liées à l'action réalisée peut être problématique.

```
const Parent = () => {
  const [count, setCount] = useState(0);
  const [countHandle, setCountHandle] = useState(0);

  const handleClick = () => {
    setCountHandle(countHandle + 1);
  };

  // const handleClick = useCallback(() => {
  //   setCountHandle(countHandle + 1);
  // }, [countHandle]);

  useEffect(() => {
    console.log("fonction recréée")
  }, [handleClick]);

  return (
    <>
      <button onClick={() => setCount(c => c + 1)}>Incrémenter count</button>
      <button onClick={handleClick}>Incrémenter countHandle</button>
    </>
  );
};
```

Dans le code ci-dessus, on pourrait penser au premier abord que lorsqu'on clique sur le bouton « Incrémenter count », le texte « fonction recréée » ne s'affichera pas, puisque **handleClick** n'a pas changé. Pourtant, ce n'est pas le cas : tout le composant se re-render et le **useEffect** se lance.

Pour éviter cela, c'est là qu'intervient **useCallback**, que l'on va appliquer à notre fonction **handleClick**. Ainsi, lorsque **count** est modifié, **handleClick** ne sera pas recréée tant que ses dépendances (ici **countHandle**) ne changent pas, ce qui évite de relancer inutilement le **useEffect** et d'afficher « fonction recréée ».

Cependant, lorsque **handleClick** est exécutée, **setCount** modifie **countHandle**, ce qui entraîne la recréation de **handleClick** et donc la réexécution du **useEffect**.

Cela permet donc un contrôle fin de la recreation des fonctions. Il faut néanmoins utiliser **useCallback** principalement sur des fonctions lourdes ou dans des cas où la performance est critique, car ce hook lui-même consomme des ressources.

## Lazy loading

Restons sur des notions moins connues de React et parlons de **React.lazy**. Ce dernier permet de **charger un composant de façon dynamique**, uniquement lorsqu'il est nécessaire de l'afficher. Cela permet de **réduire le temps de chargement** initial de la page et donc **d'améliorer ses performances**.

Le cas d'usage le plus courant concerne les **modales**, ces pop-ups au centre de l'écran affichant ou demandant des données. Concrètement, on n'a pas besoin de charger la modale si on ne l'affiche pas.

```
import React, { useState, Suspense } from 'react';

const LazyModal = React.lazy(() => import('./Modal'));

function App() {
  const [showModal, setShowModal] = useState(false);

  return (
    <div>
      <button onClick={() => setShowModal(true)}>Ouvrir la modale</button>
      {showModal && (
        <Suspense fallback={<div>Chargement de la modale...</div>}>
          <LazyModal/>
        </Suspense>
      )}
    </div>
  );
}
```

Voici une implémentation simple de **React.lazy**. On charge la modale de manière fainéante. Dans le **return**, on a un bouton qui, lorsqu'on clique dessus, passe **showModal** à **true**, ce qui déclenche l'affichage du code situé après le **&&**.

Quand **Suspense** est activé, on commence à charger la modale. Pendant ce chargement, on affiche à l'écran le message « Chargement de la modale ». Une fois le chargement terminé, la modale s'affiche.

Le **gain de temps est particulièrement important** si la modale doit effectuer un **appel distant** pour récupérer des données, car on évite ainsi de charger tout cela au démarrage de la page.

## Lintier et Prettier

Depuis le début de ce mémoire, nous n'avons abordé que des aspects internes à React. Il est maintenant intéressant de voir ce que la communauté propose pour améliorer le confort de développement avec React. Nous allons présenter ici deux outils qui s'intègrent facilement dans un projet React et qui contribuent à la maintenabilité et à la qualité du code.

- **Lintier**

Un **linter** est un **outil d'analyse statique** qui détecte les **erreurs** et **mauvaises pratiques** de programmation sans exécuter le code. Une fois installé dans un projet, il permet d'imposer un code plus **strict**, **propre** et surtout **plus lisible**. Par exemple, s'il détecte une variable non utilisée ou non déclarée, il signale une erreur directement dans l'éditeur sur la ligne concernée. Pour du **Typescript**, **ESLint** est le linter le plus connu et recommandé.

- **Prettier**

Prettier, quant à lui, ne modifie pas la logique du code mais sa **mise en forme**. Il formate automatiquement le code de manière uniforme afin d'imposer un style cohérent d'écriture. L'avantage est qu'un **fichier de configuration propre** au projet permet de définir les **règles de formatage** : par exemple, le nombre maximal de caractères par ligne, le nombre d'espaces par indentation, etc.

## Conclusion

À travers ce mémoire technique, nous avons exploré les notions fondamentales de **React**.

Dans la première partie, nous avons abordé les **composants**, éléments central de React, ainsi que les **hooks**, qui introduisent une nouvelle manière de gérer l'**état** d'un composant. Nous avons également étudié des aspects plus théoriques afin de mieux comprendre le fonctionnement interne de React, comme le **DOM virtuel** qui permet une mise à jour optimisée du DOM réel, ainsi que le **cycle de vie** d'un composant, essentiel pour comprendre à quel moment un composant se re-render.

La deuxième partie s'est intéressée aux raisons qui ont conduit notre service à privilégier React plutôt que d'autres frameworks front-end comme Vue ou Angular. Ce choix s'est appuyé autant sur les compétences internes que sur des considérations de maintenabilité, de popularité de l'outil, et de pertinence pour la carrière des développeurs. Bien que Vue soit recommandé dans les standards de l'entreprise, c'est la stabilité offerte par React, soutenu par Meta, ainsi que sa large adoption dans le monde professionnel, qui ont motivé sa généralisation dans notre service.

Enfin, dans la dernière partie, nous avons exploré plusieurs **bonnes pratiques** et **concepts avancés** destinés à optimiser les performances et améliorer la qualité et la maintenabilité du code React. Cela inclut l'intérêt de concevoir des composants **réutilisables** et **purs**, la valeur ajoutée de **TypeScript** pour renforcer la lisibilité et la robustesse du code en équipe, ainsi qu'une proposition **d'architecture** claire et bien découpée. Nous avons approfondi la logique des **hooks personnalisés**, et présenté certains hooks spécifiques à l'optimisation, comme **useMemo** et **useCallback**, qui permettent de mieux contrôler les re-renders dans une application. Le concept de chargement dynamique de composants via **React.lazy** a également été abordé, dans le but de réduire le temps de chargement initial d'une page. Enfin, nous avons évoqué deux outils externes à React, **ESLint** et **Prettier**, qui viennent renforcer la rigueur et l'uniformité du code source.

L'objectif principal de ce mémoire était d'offrir une **base de compréhension** commune autour de React, à destination des membres du service, qu'ils soient débutants ou déjà expérimentés mais **en quête de meilleures pratiques**. Il s'agissait aussi de mettre en lumière des notions moins connues. Ce mémoire a permis des échanges entre développeurs autour des subtilités du framework.

À présent, il s'agit d'identifier les projets existants qui pourraient bénéficier d'optimisations, et d'exploiter les différents points présentés dans ce mémoire pour **améliorer** la performance et la maintenabilité de nos applications React à long terme.

## Bibliographie

### Origine de React

[React](#)

[React — Wikipédia](#)

### Notion de composant

[React – Une bibliothèque JavaScript pour créer des interfaces utilisateurs](#)

<https://www.linkedin.com/feed/>

### Notion de Hooks

[React – Une bibliothèque JavaScript pour créer des interfaces utilisateurs](#)

[Built-in React Hooks – React](#)

### Gestion du DOM virtuel

[DOM virtuel ReactJS - GeeksforGeeks](#)

### Cycle de vie d'un composant

[Cycle de vie des Effets réactifs – React](#)

### Pourquoi avoir choisi React

[Most used web frameworks among developers 2024| Statista](#)

[Stack Overflow Developer Survey 2022](#)

[Foire aux questions | Vue.js](#)

### Utilisation de composants réutilisables et purs

[Réutiliser de la logique grâce aux Hooks personnalisés – React](#)

[memo – React](#)

[Garder les composants purs – React](#)

### Utilisation de Typescript

[TypeScript: Documentation - Object Types](#)

[React With TypeScript Vs JavaScript: Which is Better?](#)

### Architecture d'un projet

[Best Folder Structure for Scalable React Apps: A Complete Step-by-Step Guide - DEV Community](#)

[Popular React Folder Structures and Screaming Architecture](#)

### Les hooks plus poussés

[Réutiliser de la logique grâce aux Hooks personnalisés – React](#)

[useMemo – React](#)

<https://youtu.be/6-BfMpTT2PE>

<https://youtu.be/jX57Hcq-Rv8>

[https://youtu.be/\\_AyFP5s69N4](https://youtu.be/_AyFP5s69N4)

### Lazy loading

[lazy – React](#)

[Lazy Loading in React and How to Implement it ? - GeeksforGeeks](#)

### Linter et Prettier

[Qu'est que le linting et pourquoi linter votre code ? - Mindsers Blog](#)

[Find and fix problems in your JavaScript code - ESLint - Pluggable JavaScript Linter](#)

[Prettier · Opinionated Code Formatter · Prettier](#)